UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas

ENTREGABLE FINAL - PROGRAMACIÓN IV

SISTEMA WEB DE GESTIÓN DE INVENTARIO PARA LA LIBRERÍA PUNTO Y COMA CON INTELIGENCIA ARTIFICIAL LOCAL MEDIANTE DJANGO Y OLLAMA

Estudiante: Renzo Joaquin Diaz Zumelzu

Docente: Jared Lopez Leaños

Asignatura: Programación IV

Modalidad: Individual

Sede: Santa Cruz de la Sierra

Gestión: 2026

Informe técnico del proyecto

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación web para la gestión del inventario de una librería ficticia denominada Librería Punto y Coma.

La aplicación fue desarrollada utilizando Django como framework web, SQLite como sistema de base de datos y Ollama para ejecutar un modelo de inteligencia artificial de manera local.

El sistema permite administrar productos mediante operaciones CRUD, controlar las cantidades existentes, realizar búsquedas y filtros, generar reportes del inventario y realizar consultas mediante lenguaje natural utilizando inteligencia artificial.

Como característica adicional, la aplicación implementa un mecanismo de restricciones para que la inteligencia artificial responda únicamente preguntas relacionadas con los productos registrados en el sistema.

Las consultas realizadas mediante inteligencia artificial también son almacenadas en un historial, permitiendo comprobar las preguntas realizadas, las respuestas generadas y los posibles errores de comunicación con Ollama.

El proyecto fue implementado y probado en un entorno Linux MiniOS utilizando Python 3.11 y Django 5.0.4.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación web utilizando Django que permita administrar el inventario de una librería e integrar un modelo de inteligencia artificial ejecutado localmente mediante Ollama para realizar consultas en lenguaje natural sobre los productos almacenados.

2.2. Objetivos específicos

- Implementar operaciones CRUD para administrar productos.
- Utilizar SQLite como base de datos del sistema.
- Validar la información ingresada antes de almacenarla.
- Controlar las cantidades existentes de cada producto.
- Implementar búsquedas y filtros de productos.
- Generar reportes automáticos del inventario.
- Integrar Django con la API local de Ollama.
- Utilizar un modelo de inteligencia artificial ligero compatible con los recursos disponibles.
- Restringir las consultas de inteligencia artificial al dominio del inventario.
- Evitar que el modelo utilice información externa para responder consultas.
- Registrar un historial de las consultas realizadas mediante inteligencia artificial.
- Permitir que el proyecto pueda instalarse y ejecutarse nuevamente mediante instrucciones documentadas.

3. Descripción del caso de estudio

3.1. Librería Punto y Coma

Para proporcionar un contexto concreto al sistema se definió como caso de estudio una empresa ficticia denominada Librería Punto y Coma.

El negocio administra productos correspondientes principalmente a:

- Libros de literatura.
- Libros académicos.
- Material infantil.
- Cuadernos.
- Material de escritura.
- Material de oficina.
- Otros productos relacionados.

La aplicación se diseñó visual y funcionalmente considerando este tipo de negocio.

3.2. Problema identificado

Una librería necesita conocer de manera rápida información relacionada con los productos que mantiene disponibles.

Entre las operaciones necesarias se encuentran:

- Registrar nuevos productos.
- Modificar información existente.
- Consultar productos.
- Eliminar registros.
- Controlar las existencias.
- Detectar productos agotados.
- Detectar productos con poco stock.
- Conocer el valor del inventario.
- Identificar los productos de mayor o menor precio.

Normalmente estas operaciones se realizan utilizando formularios, tablas y reportes.

En este proyecto se plantea complementar dichas herramientas con una interfaz de inteligencia artificial que permita realizar consultas utilizando lenguaje natural.

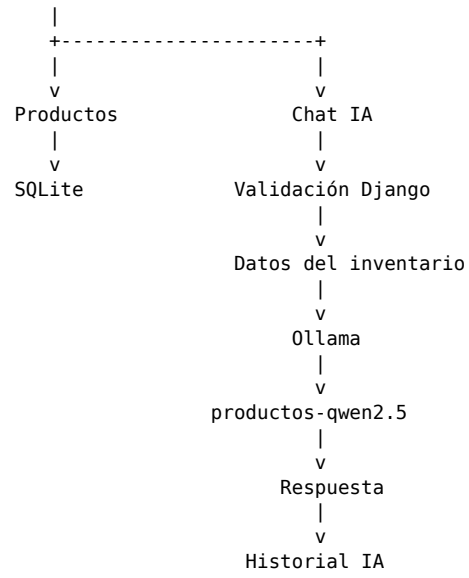
3.3. Solución propuesta

Se desarrolló una aplicación Django compuesta principalmente por dos módulos:

1. Módulo de productos, encargado de administrar el inventario.
2. Módulo de inteligencia artificial, encargado de comunicarse con Ollama.

La arquitectura general puede representarse de la siguiente manera:

```
Usuario
   |
   v
Interfaz web Django
```



4. Tecnologías utilizadas

Las principales tecnologías utilizadas fueron:

Tecnología	Utilización
Python 3.11.2	Lenguaje de programación
Django 5.0.4	Framework web
SQLite	Base de datos
Ollama	Ejecución local de modelos de IA
Qwen2.5 1.5B	Modelo base
productos-qwen2.5	Modelo personalizado
HTML5	Estructura de las páginas
CSS3	Diseño de la interfaz
JavaScript	Interacciones de interfaz
Git	Control de versiones
MiniOS	Sistema operativo utilizado durante el desarrollo

Se seleccionó SQLite debido a que el proyecto corresponde a una aplicación académica y no requiere instalar un servidor independiente de base de datos.

Ollama permite ejecutar la inteligencia artificial localmente, evitando depender de servicios de IA externos.

5. Arquitectura del sistema

Django permite separar las responsabilidades de la aplicación mediante modelos, vistas, formularios, rutas y plantillas.

El proyecto utiliza principalmente las aplicaciones:

```
productos
```

```
chat
```

La aplicación `productos` contiene la lógica relacionada con:

- Productos.
- Formularios.
- CRUD.
- Dashboard.
- Existencias.
- Reportes.
- Carga de datos de demostración.

La aplicación `chat` contiene:

- Comunicación con Ollama.
- Validación de preguntas.
- Preparación del inventario.
- Registro de consultas.
- Historial de inteligencia artificial.

La configuración general se encuentra en:

```
chatapp/
```

6. Desarrollo del módulo CRUD

6.1. Modelo Producto

El inventario utiliza la entidad `Producto`.

El modelo contiene los siguientes campos:

Campo	Tipo	Función
id	Entero	Identificador automático
codigo	Texto	Código único
nombre	Texto	Nombre del producto
descripcion	Texto	Descripción
categoria	Texto	Categoría
precio	Decimal	Precio
cantidad_existente	Entero positivo	Existencias
stock_minimo	Entero positivo	Nivel mínimo
estado	Booleano	Activo/inactivo
fecha_registro	Fecha y hora	Fecha de creación

Las categorías definidas son:

```
class Categoria(models.TextChoices):
    LITERATURA = "literatura", "Literatura"
    ACADEMICOS = "academicos", "Académicos"
    INFANTIL = "infantil", "Infantil"
    CUADERNOS = "cuadernos", "Cuadernos"
```

```
    ESCRITURA = "escritura", "Escritura"
    OFICINA = "oficina", "Oficina"
    OTROS = "otros", "Otros"
```

El código se define como único:

```
codigo = models.CharField(
    max_length=20,
    unique=True,
    verbose_name="Código",
)
```

El precio utiliza un validador para impedir números negativos:

```
precio = models.DecimalField(
    max_digits=10,
    decimal_places=2,
    validators=[MinValueValidator(0)],
    verbose_name="Precio",
)
```

También se implementaron propiedades auxiliares.

Producto agotado:

```
@property
def agotado(self):
    return self.cantidad_existente == 0
```

Stock bajo:

```
@property
def stock_bajo(self):
    return (
        self.cantidad_existente > 0
        and self.cantidad_existente <= self.stock_minimo
    )
```

Valor del producto dentro del inventario:

```
@property
def valor_inventario(self):
    return self.precio * self.cantidad_existente
```

7. Validación de formularios

Se desarrolló `ProductoForm` utilizando `ModelForm`.

Además de las restricciones definidas en el modelo, se añadieron validaciones en el formulario.

7.1. Código único

Antes de guardar un código se convierte a mayúsculas:

```
codigo = self.cleaned_data["codigo"].strip().upper()
```

Posteriormente se comprueba que no exista otro producto con el mismo código:

```
consulta = Producto.objects.filter(codigo__iexact=codigo)

if self.instance.pk:
    consulta = consulta.exclude(pk=self.instance.pk)
```

```
if consulta.exists():
    raise forms.ValidationError(
        "Ya existe otro producto con este código."
    )
```

7.2. Validación del precio

```
if precio < 0:
    raise forms.ValidationError(
        "El precio no puede ser negativo."
    )
```

7.3. Validación de existencias

```
if cantidad < 0:
    raise forms.ValidationError(
        "La cantidad existente no puede ser negativa."
    )
```

La misma lógica se utiliza para impedir un stock mínimo negativo.

8. Operaciones CRUD

8.1. Crear

El registro utiliza `ProductoForm`.

Cuando el formulario es válido se almacena el producto:

```
if formulario.is_valid():
    producto = formulario.save()
```

Posteriormente Django muestra un mensaje indicando que la operación se realizó correctamente.

8.2. Consultar

La aplicación dispone de:

- Listado general.
- Vista individual.
- Búsqueda.
- Filtros por categoría.
- Filtros por estado de stock.

La búsqueda utiliza objetos `Q`:

```
productos = productos.filter(
    Q(codigo__icontains=busqueda)
    | Q(nombre__icontains=busqueda)
    | Q(descripcion__icontains=busqueda)
)
```

Esto permite buscar utilizando código, nombre o descripción.

8.3. Actualizar

La edición reutiliza el formulario existente:

```
formulario = ProductoForm(
    request.POST,
    instance=producto,
)
```

De esta manera se mantienen las mismas validaciones utilizadas durante el registro.

8.4. Eliminar

Antes de eliminar un producto se muestra una página de confirmación.

La eliminación finalmente se realiza mediante:

```
producto.delete()
```

Después se muestra un mensaje de confirmación.

9. Control de existencias

Además de editar manualmente la cantidad, el sistema dispone de operaciones para aumentar y disminuir las existencias.

Para aumentar:

```
producto.cantidad_existente += 1
producto.save()
```

Para disminuir se verifica primero que exista stock:

```
if producto.cantidad_existente <= 0:
    messages.error(
        request,
        "No se puede disminuir el stock porque ya está agotado."
    )
else:
    producto.cantidad_existente -= 1
    producto.save()
```

Esta comprobación evita obtener cantidades negativas.

10. Base de datos y datos de demostración

El sistema utiliza SQLite durante el desarrollo.

Después de implementar el modelo se generaron y aplicaron las migraciones:

```
python manage.py makemigrations productos
python manage.py migrate
```

La migración fue aplicada correctamente:

```
Applying productos.0001_initial... OK
```

Posteriormente se comprobó que SQLite contenía la tabla:

```
productos_producto
```

Para facilitar las demostraciones se creó el comando:

```
python manage.py cargar_datos_demo
```

El comando carga productos correspondientes a Librería Punto y Coma.

Resultado obtenido:

```
Productos creados: 12
Productos existentes: 0
Total en la base de datos: 12
Datos de demostración cargados correctamente.
```

Al ejecutar nuevamente el comando:

```
Productos creados: 0
Productos existentes: 12
Total en la base de datos: 12
```

Por lo tanto, el proceso evita duplicar los productos de demostración.

Entre los productos registrados se encuentran:

```
LIB001 - El principito
LIB002 - Cien años de soledad
LIB003 - Álgebra de Baldor
LIB004 - Don Quijote de la Mancha
LIB005 - Cuentos de la selva
PAP001 - Cuaderno universitario
PAP002 - Bolígrafo azul
PAP003 - Lápiz HB
PAP004 - Marcadores de colores
OFI001 - Resma de papel A4
OFI002 - Archivador tamaño carta
OFI003 - Carpeta plástica
```

11. Sistema de reportes

La aplicación implementa ocho reportes.

11.1. Todos los productos

```
todos_productos = productos.order_by("nombre")
```

11.2. Producto más caro

```
producto_mas_caro = productos.order_by(
    "-precio"
).first()
```

Durante las pruebas se obtuvo:

```
LIB004 - Don Quijote de la Mancha
Precio: Bs 120,00
```

11.3. Producto más barato

```
producto_mas_barato = productos.order_by(
    "precio"
).first()
```

Resultado comprobado:

```
PAP003 - Lápiz HB
```

11.4. Productos con stock bajo

La consulta compara la cantidad disponible con el stock mínimo:

```
stock_bajo = productos.filter(
    cantidad_existente__gt=0,
    cantidad_existente__lte=F("stock_minimo"),
)
```

11.5. Productos agotados

```
agotados = productos.filter(
    cantidad_existente=0
)
```

Durante las pruebas se detectó:

```
OFI001 - Resma de papel A4
```

como producto agotado.

11.6. Productos por categoría

El usuario puede seleccionar una categoría y Django filtra los registros correspondientes.

Ejemplo comprobado para literatura:

```
Cien años de soledad
Don Quijote de la Mancha
El principito
```

11.7. Valor total del inventario

El cálculo se realiza desde la base de datos utilizando `Sum`, `F` y `ExpressionWrapper`.

```
valor_total = productos.aggregate(
    total=Sum(
        ExpressionWrapper(
            F("precio") * F("cantidad_existente"),
            output_field=DecimalField(
                max_digits=14,
                decimal_places=2,
            ),
        )
    )
)["total"] or 0
```

Con los datos de demostración se obtuvo:

```
Valor total del inventario: Bs 3597
```

11.8. Productos con mayor existencia

```
mayor_existencia = productos.order_by(
    "-cantidad_existente",
    "nombre"
```

```
)[:5]
```

Esto permite mostrar los cinco productos con mayor cantidad disponible.

12. Integración con inteligencia artificial local

12.1. Ollama

Para integrar inteligencia artificial se utilizó Ollama.

Durante las pruebas se comprobó la instalación:

```
ollama --version
```

Resultado:

```
ollama version is 0.34.0
```

Los modelos instalados inicialmente fueron:

```
gemma4:cloud
qwen2.5:1.5b
```

Para el proyecto se seleccionó:

```
qwen2.5:1.5b
```

Este modelo ocupa aproximadamente 986 MB, por lo que resulta adecuado para el entorno disponible.

13. Prueba inicial de Ollama

Antes de integrarlo con Django se comprobó directamente la API local:

```
curl -s http://127.0.0.1:11434/api/generate \
  -d '{"model":"qwen2.5:1.5b","prompt":"Responde solamente: FUNCIONA","stream":false}'
```

El modelo respondió:

```
FUNCIONA
```

Con esta prueba se verificó que:

- Ollama estaba instalado.
- El servicio estaba disponible.
- El modelo podía cargarse.
- La API local respondía correctamente.

14. Modelo personalizado

Para adaptar el comportamiento del modelo al sistema se creó:

```
ollama/Modelfile-productos
```

El modelo parte de:

```
FROM qwen2.5:1.5b
```

Se configuraron los siguientes parámetros:

```
PARAMETER temperature 0.1
PARAMETER top_p 0.8
PARAMETER num_ctx 4096
PARAMETER num_predict 300
PARAMETER repeat_penalty 1.1
```

La temperatura baja busca favorecer respuestas más consistentes.

El SYSTEM define al modelo como:

```
Asistente de Inventario de la Librería Punto y Coma
```

y establece que su única fuente válida deben ser los datos del inventario enviados por el sistema.

El modelo personalizado fue creado mediante:

```
ollama create productos-qwen2.5 -f ollama/Modelfile-productos
```

El resultado fue:

```
writing manifest
success
```

Posteriormente:

```
ollama list
```

mostró:

```
productos-qwen2.5:latest
qwen2.5:1.5b
gemma4:cloud
```

15. Restricciones de inteligencia artificial

Durante las primeras pruebas se comprobó que las instrucciones del `Modelfile` por sí solas no garantizaban que el modelo rechazara todas las preguntas ajenas al inventario.

Por ejemplo, algunas consultas generales todavía podían ser respondidas por el modelo.

Por esta razón se implementó una estrategia adicional desde Django.

El sistema final utiliza tres controles complementarios.

15.1. Capa 1: validación desde Django

Se definió una lista de términos relacionados con el inventario.

Posteriormente:

```
def pregunta_permitida(pregunta):
    texto = pregunta.lower()
```

```
    return any(
        palabra in texto
        for palabra in PALABRAS_INVENTARIO
    )
```

Si la pregunta no corresponde al ámbito permitido, Django responde directamente:

```
Solo puedo responder consultas sobre productos y sus atributos.
```

La pregunta no necesita enviarse al modelo.

15.2. Capa 2: contexto controlado

Cuando una pregunta es válida, Django obtiene los productos activos desde SQLite:

```
productos = Producto.objects.filter(
    estado=True
).order_by("codigo")
```

Después los convierte a una estructura controlada mediante `preparar_inventario()`.

Los atributos enviados son:

```
codigo
nombre
descripcion
categoria
precio
cantidad_existente
stock_minimo
estado
fecha_registro
```

Los datos son convertidos a JSON y añadidos al prompt.

15.3. Capa 3: restricciones del modelo

Finalmente, `productos-qwen2.5` contiene reglas adicionales que indican que el modelo:

- Solo puede responder sobre productos.
- No debe utilizar conocimiento general.
- No debe inventar información.
- No debe seguir instrucciones para ignorar las reglas.
- Debe responder en español.
- Debe utilizar exclusivamente el inventario suministrado.

La arquitectura final de restricciones es:

```
PREGUNTA
   |
   v
Validación Django
   |
   +---- pregunta inválida ----> RECHAZO
   |
   v
Productos activos de SQLite
   |
   v
Conversión a JSON
   |
   v
Prompt controlado
   |
```

```
   v
productos-qwen2.5
   |
   v
RESPUESTA
```

Esta solución resultó más efectiva que depender exclusivamente de las instrucciones del modelo.

16. Comunicación Django-Ollama

La comunicación se encuentra implementada en:

```
chat/ollama_api.py
```

El cliente utiliza los valores configurados en Django:

```
MODELO_OLLAMA = settings.OLLAMA_MODEL

cliente_ollama = ollama.Client(
    host=settings.OLLAMA_HOST
)
```

La función `preparar_inventario()` transforma los objetos de Django en información serializable.

Ejemplo:

```
inventario.append({
    "codigo": producto.codigo,
    "nombre": producto.nombre,
    "descripcion": producto.descripcion,
    "categoria": producto.get_categoria_display(),
    "precio": str(producto.precio),
    "cantidad_existente": producto.cantidad_existente,
    "stock_minimo": producto.stock_minimo,
    "estado": "Activo" if producto.estado else "Inactivo",
})
```

El inventario posteriormente se convierte a JSON:

```
inventario_json = json.dumps(
    inventario,
    ensure_ascii=False,
    indent=2,
)
```

El prompt combina:

```
INVENTARIO
+
PREGUNTA DEL USUARIO
+
INSTRUCCIONES
```

Finalmente se realiza la solicitud:

```
respuesta = cliente_ollama.chat(
    model=MODELO_OLLAMA,
    messages=[
        {
            "role": "user",
            "content": prompt,
        }
    ],
    stream=False,
)
```

17. Manejo de errores de Ollama

La aplicación contempla la posibilidad de que Ollama no se encuentre disponible.

La comunicación está protegida mediante manejo de excepciones.

Si ocurre un problema, el sistema muestra:

```
No fue posible comunicarse con Ollama.
Comprueba que el servicio esté disponible.
```

De esta forma un fallo en el servicio de inteligencia artificial no produce una página de error sin controlar.

Además, el problema queda registrado en el historial.

18. Historial de consultas de inteligencia artificial

Para mantener evidencia de las consultas se implementó el modelo:

```
ConsultaIA
```

Sus principales campos son:

```
pregunta
respuesta
fecha
modelo
exitosa
tipo_resultado
```

Los resultados se clasifican en:

```
TIPOS_RESULTADO = [
    ("ia", "Procesada por IA"),
    ("rechazada", "Rechazada por ámbito"),
    ("error", "Error de Ollama"),
]
```

Por lo tanto, el historial permite diferenciar:

- Consultas procesadas por inteligencia artificial.
- Consultas rechazadas por las restricciones.
- Errores de comunicación con Ollama.

Cuando una consulta es procesada correctamente:

```
ConsultaIA.objects.create(
    pregunta=pregunta,
    respuesta=respuesta,
    modelo=MODELO_OLLAMA,
    exitosa=True,
    tipo_resultado="ia",
)
```

Cuando Django rechaza una pregunta:

```
ConsultaIA.objects.create(
    pregunta=pregunta,
    respuesta=respuesta,
```

```
    modelo="Validación Django",
    exitosa=True,
    tipo_resultado="rechazada",
)
```

También se almacenan los errores:

```
ConsultaIA.objects.create(
    pregunta=pregunta,
    respuesta=error,
    modelo=MODELO_OLLAMA,
    exitosa=False,
    tipo_resultado="error",
)
```

Esto proporciona trazabilidad sobre el uso del módulo de inteligencia artificial.

19. Variables de entorno

Los valores dependientes de cada equipo se manejan mediante variables de configuración.

El proyecto incluye:

```
.env.example
```

como referencia.

Entre las configuraciones se encuentran:

```
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost,IP_DE_LA_MAQUINA
OLLAMA_HOST=http://127.0.0.1:11434
OLLAMA_MODEL=productos-qwen2.5
```

El archivo real:

```
.env
```

se encuentra excluido del repositorio mediante `.gitignore`.

Esto permite modificar las direcciones o el modelo sin alterar directamente el código fuente.

20. Ejecución del sistema

El entorno virtual se activa mediante:

```
source venv/bin/activate
```

Las dependencias se instalan con:

```
pip install -r requirements.txt
```

El archivo contiene:

```
Django==5.0.4
ollama==0.6.2
python-dotenv==1.0.1
```

Posteriormente:

```
python manage.py migrate
python manage.py cargar_datos_demo
```

Para iniciar Django:

```
python manage.py runserver 0.0.0.0:8000
```

El servidor confirmó:

```
Django version 5.0.4, using settings 'chatapp.settings'
Starting development server at http://0.0.0.0:8000/
```

21. Acceso desde la red local

El uso de:

```
python manage.py runserver 0.0.0.0:8000
```

permite escuchar solicitudes desde las interfaces de red disponibles.

Durante las pruebas se comprobó localmente:

```
curl -I http://127.0.0.1:8000/productos/
```

Resultado:

```
HTTP/1.1 200 OK
```

La ruta principal también fue comprobada:

```
curl -I http://127.0.0.1:8000/
```

obteniendo:

```
HTTP/1.1 302 Found
Location: /productos/
```

Esto confirma que la raíz redirecciona correctamente hacia la aplicación de productos.

Para utilizar el sistema desde otro equipo se debe ingresar:

```
http://IP_DEL_SERVIDOR:8000/
```

y configurar la dirección correspondiente dentro de los hosts permitidos.

22. Pruebas realizadas

Durante el desarrollo se realizaron diferentes pruebas funcionales.

22.1. Comprobación de Django

```
python manage.py check
```

Resultado:

```
System check identified no issues (0 silenced).
```

22.2. Migraciones

```
python manage.py showmigrations productos
```

Resultado:

```
productos
 [X] 0001_initial
```

22.3. Cantidad de productos

```
Producto.objects.count()
```

Resultado:

```
12
```

22.4. Producto más caro

```
Producto.objects.order_by("-precio").first()
```

Resultado:

```
LIB004 - Don Quijote de la Mancha
```

Precio:

```
120.00
```

22.5. Producto más barato

```
Producto.objects.order_by("precio").first()
```

Resultado:

```
PAP003 - Lápiz HB
```

22.6. Productos agotados

```
Producto.objects.filter(cantidad_existente=0)
```

Resultado:

```
OFI001 - Resma de papel A4
```

22.7. Productos de literatura

```
Producto.objects.filter(categoria="literatura")
```

Resultado:

```
Cien años de soledad
Don Quijote de la Mancha
El principito
```

22.8. Productos con pocas existencias

La consulta:

```
Producto.objects.filter(
    cantidad_existente__lte=F("stock_minimo")
)
```

detectó registros con existencias iguales o inferiores al nivel mínimo configurado.

22.9. Valor del inventario

Se calculó mediante:

```
F("precio") * F("cantidad_existente")
```

Resultado:

```
Bs 3597
```

22.10. Ollama

La API respondió correctamente utilizando:

```
qwen2.5:1.5b
```

Posteriormente se creó:

```
productos-qwen2.5
```

22.11. Restricciones

Se probaron preguntas ajenas al inventario.

La implementación final permite que Django rechace dichas consultas antes de enviarlas a Ollama.

Respuesta esperada:

```
Solo puedo responder consultas sobre productos y sus atributos.
```

También se comprobaron preguntas válidas relacionadas con:

- Precios.
- Existencias.
- Categorías.
- Productos agotados.
- Productos disponibles.
- Datos del inventario.

23. Estructura final del proyecto

La estructura principal es:

```
ollama-django/
```

```
|
+-- chat/
|   +-- migrations/
|   +-- templates/
|   |   +-- chat.html
|   |   +-- historial.html
|   +-- admin.py
|   +-- apps.py
|   +-- models.py
|   +-- ollama_api.py
|   +-- views.py
|
+-- chatapp/
|   +-- settings.py
|   +-- urls.py
|   +-- asgi.py
|   +-- wsgi.py
|
+-- productos/
|   +-- management/
|   |   +-- commands/
|   |       +-- cargar_datos_demo.py
|   |
|   +-- migrations/
|   +-- templates/
|   |   +-- productos/
|   |       +-- base.html
|   |       +-- dashboard.html
|   |       +-- detalle.html
|   |       +-- eliminar.html
|   |       +-- formulario.html
|   |       +-- lista.html
|   |       +-- reportes.html
|   |
|   +-- admin.py
|   +-- apps.py
|   +-- forms.py
|   +-- models.py
|   +-- urls.py
|   +-- views.py
|
+-- ollama/
|   +-- Modelfile-productos
|
+-- .env.example
+-- .gitignore
+-- DOCUMENTACION.md
+-- README.md
+-- requirements.txt
+-- manage.py
```

24. Reproducibilidad del proyecto

Uno de los objetivos de la preparación final fue evitar que el proyecto dependiera del entorno virtual utilizado durante el desarrollo.

Por este motivo no es necesario distribuir:

```
venv/
```

El entorno puede reconstruirse mediante:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Tampoco es necesario incluir físicamente el modelo de aproximadamente 986 MB dentro del proyecto.

Puede recuperarse mediante:

```
ollama pull qwen2.5:1.5b
```

y posteriormente crear el modelo personalizado:

```
ollama create productos-qwen2.5 -f ollama/Modelfile-productos
```

Esto permite mantener el proyecto y el archivo comprimido con un tamaño reducido.

25. Dificultades encontradas

Durante el desarrollo se presentaron diferentes dificultades técnicas.

25.1. Configuración inicial del proyecto

El proyecto original únicamente contenía una aplicación básica de chat conectada a Ollama y no disponía del módulo de productos requerido.

Fue necesario incorporar:

- Aplicación `productos`.
- Modelo de datos.
- Migraciones.
- Formularios.
- CRUD.
- Reportes.
- Dashboard.
- Historial.

25.2. Modelo utilizado inicialmente

La configuración original hacía referencia a:

```
llama3
```

pero dicho modelo no se encontraba instalado.

Se comprobó la disponibilidad real mediante:

```
ollama list
```

y se seleccionó:

```
qwen2.5:1.5b
```

25.3. Restricciones del modelo

Una de las principales dificultades fue restringir al modelo únicamente al ámbito del inventario.

Las instrucciones configuradas mediante `Modelfile` no fueron suficientes durante las primeras pruebas.

Por esta razón se añadió una validación en Django.

La experiencia permitió comprobar que las instrucciones de un modelo generativo no deben considerarse una restricción absoluta.

25.4. Configuración de red

La máquina MiniOS utilizada durante el desarrollo disponía de varias interfaces de red.

Se realizaron pruebas para permitir el acceso a Django desde otros dispositivos utilizando:

```
python manage.py runserver 0.0.0.0:8000
```

Las pruebas permitieron verificar que Django podía escuchar conexiones externas cuando la configuración de red y los hosts permitidos eran correctos.

25.5. Errores durante el desarrollo

También se detectaron y corrigieron errores relacionados con:

- Indentación en `views.py`.
- Referencias incorrectas a las categorías.
- Configuración de rutas.
- Migraciones pendientes.
- Restricciones insuficientes del modelo.

Cada problema fue corregido y posteriormente se utilizó:

```
python manage.py check
```

para comprobar el estado del proyecto.

26. Limitaciones del sistema

El sistema presenta algunas limitaciones propias de su alcance académico.

26.1. SQLite

SQLite resulta adecuado para desarrollo y demostraciones, pero una aplicación empresarial con alta concurrencia podría requerir un sistema como PostgreSQL.

26.2. Servidor de desarrollo

El comando:

```
python manage.py runserver
```

no constituye un servidor destinado a producción.

26.3. Modelo pequeño

`qwen2.5:1.5b` fue seleccionado por su reducido consumo de recursos.

Como consecuencia, su capacidad de interpretación es menor que la de modelos de mayor tamaño.

26.4. Validación por palabras

La validación previa de Django utiliza palabras relacionadas con el inventario.

Esto resulta apropiado para el alcance de la práctica, aunque un sistema más avanzado podría utilizar mecanismos de clasificación de intención más sofisticados.

26.5. Alcance empresarial

El sistema no incluye:

- Ventas.
- Compras.
- Facturación.
- Proveedores.
- Clientes.
- Contabilidad.

Estas funciones se encuentran fuera del alcance establecido.

27. Reflexión técnica

El desarrollo permitió integrar dos áreas diferentes de programación.

Por una parte, Django proporciona un sistema determinista para administrar información mediante modelos, formularios, consultas y vistas.

Por otra parte, Ollama incorpora un modelo generativo que permite consultar los mismos datos utilizando lenguaje natural.

Una de las conclusiones técnicas más importantes obtenidas durante el desarrollo fue que la inteligencia artificial no debe ser utilizada como sustituto de las reglas del sistema.

Por ejemplo, para determinar el producto más caro, Django puede realizar directamente:

```
productos.order_by("-precio").first()
```

mientras que la inteligencia artificial resulta útil para explicar los resultados de una manera comprensible para el usuario.

También se comprobó que las restricciones escritas dentro del prompt o `Modelfile` pueden ser insuficientes.

Por este motivo, el sistema utiliza validación desde Django antes de consultar al modelo.

Esta separación permite que las reglas importantes permanezcan bajo control de la aplicación y que la inteligencia artificial se utilice principalmente como una interfaz de consulta en lenguaje natural.

28. Conclusiones

Se desarrolló una aplicación web funcional para la administración del inventario de la Librería Punto y Coma utilizando Django.

El sistema implementa las operaciones fundamentales de creación, consulta, actualización y eliminación de productos.

La utilización del ORM de Django permitió realizar búsquedas, filtros, cálculos y reportes directamente sobre SQLite.

Se implementó un conjunto de datos de demostración reproducible que permite preparar rápidamente el sistema para una presentación.

La integración con Ollama permitió incorporar consultas mediante lenguaje natural sin utilizar un servicio externo de inteligencia artificial.

Se creó un modelo personalizado denominado `productos-qwen2.5` basado en `qwen2.5:1.5b`.

Durante las pruebas se determinó que las restricciones del modelo por sí solas no eran suficientes, por lo que se incorporó una validación previa desde Django.

La solución final utiliza el inventario almacenado en SQLite como fuente de información para las consultas enviadas a Ollama.

Además, el historial implementado permite conservar las preguntas y respuestas generadas, incluyendo consultas rechazadas y errores de comunicación.

Finalmente, el proyecto fue preparado para poder ser reproducido en otro equipo mediante `requirements.txt`, `.env.example`, migraciones, datos de demostración, `Modelfile-productos`, README y documentación técnica.

29. Referencias

Django Software Foundation. (2026). Django documentation. Documentación oficial del framework Django.

Ollama. (2026). Ollama documentation. Documentación oficial de Ollama.

Python Software Foundation. (2026). Python documentation. Documentación oficial del lenguaje Python.

SQLite. (2026). SQLite documentation. Documentación oficial de SQLite.

Qwen Team. (2026). Qwen2.5. Documentación técnica del modelo utilizado como base.

30. Anexo: comandos principales

Preparación

```
cd ~/ollama-django

python3 -m venv venv
source venv/bin/activate

pip install -r requirements.txt
```

Configuración

```
cp .env.example .env
```

Base de datos

```
python manage.py migrate
python manage.py cargar_datos_demo
```

Ollama

```
ollama pull qwen2.5:1.5b

ollama create productos-qwen2.5 \
    -f ollama/Modelfile-productos

ollama list
```

Comprobación de Django

```
python manage.py check
```

Ejecución

```
python manage.py runserver 0.0.0.0:8000
```

Acceso local

```
http://127.0.0.1:8000/
```

Rutas principales

```
/productos/
/productos/reportes/
/chat/
```

Fin del informe